ST. ANTONY'S COLLEGE

PERUVANTHANAM

AFFILIATED TO MAHATMA GANDHI UNIVERSITY

BACHELOR OF COMPUTER APPLICATIONS

NAME:…….

CLASS:……SEMESTER……...

SUBJECT:……

YEAR:……..……REG.NO:…….

ST. ANTONY'S COLLEGE

PERUVANTHANAM

AFFILIATED TO MAHATMA GANDHI UNIVERSITY

CERTIFICATE

This is to certify that this Record work done in .....by Mr./Ms..... during the year .....in partial fulfillment of the requirement for the award of the degree of .....of Mahatma Gandhi University, Kottayam, Kerala.

Semester:……. Reg. No:……

Monisha M Mohanan Jintumol John

(Faculty-in Charge) (Dept. Seal) (Head of the Department)

Submitted for the Practical Examination held on……..

Internal Examiner **External Examiner**

INDEX

SL. NO	PROGRAM	PAGE NUMBER	DATE	REMARKS
1	Tokenize a Paragraph into Sentences and Words.	1	26-06-2026	
2	Build Unigram Language Model for a Given Corpus.	2	03-07-2026	
3	Rule-Based System to Identify Parts of a Sentence.	3	03-07-2026	
4	Basic Semantic Analysis Using Word Similarity.	4	10-07-2026	
5	Demonstrate Pragmatics by Identifying Speech Acts	5	10-07-2026	
6	Use Python libraries to analysis speech signals.	6	17-07-2026	
7	Convert text to phoneme representation.	7	17-07-2026	
8	Demonstrate ASCII Vs Unicode Encoding.	8	24-07-2026	
9	Handle Multilingual Text Input and Normalization	9	24-07-2026	
10	Implementation of Text Preprocessing	10	13-08-2026	
11	Load and explore a corpus using NLTK.	12	13-08-2026	
12	Access and query WordNet for synonyms, antonyms and hypernyms.	13	18-08-2026	
13	Parse and extract information from an XML based linguistic resource.	15	18-08-2026	
14	Using regx patterns to extract emails, dates, and URLs from text.	17	11-09-2026	

15	Simple Finite State Automation (FSA) for Word Recognition.	18	11-09-2026	
16	Simple Morphological Analyzer (Prefix, Suffix Stripping).	19	14-09-2026	
17	Perform POS tagging using NLTK.	21	14-09-2026	
18	Stochastic POS Tagger using Hidden Markov Model.	22	18-09-2026	
19	Transformation Based Learning (TBL) Tagging.	24	18-09-2026	
20	Handle Unknown Words in POS Tagging.	26	23-09-2026	
21	Define a Context Free Grammer (CFG) and Parse Sentences.	28	23-09-2026	
22	Use Chart Parsing to Analyze Ambiguous Sentences.	30	28-09-2026	
23	Load and Analyze Treebank Using NLTK.	32	28-09-2026	
24	Represent Sentence Meaning Using Logical Forms.	34	05-10-2026	
25	Simple Machine Translation System Using Sequence Models.	35	05-10-2026	

Program No: 1

Aim

Write a Python program to tokenize a paragraph into sentences and words using NLTK.

Program

```
import nltk
nltk.download('punkt_tab')
from nltk.tokenize import sent_tokenize,word_tokenize
text=input("enter the paragraph:")
print("sentence tokenization:")
print(sent_tokenize(text))
print("\nword tokenization:")
print(word_tokenize(text))
```

Output

```
>>>
====================== RESTART: D:\MMM\NLP Lab\Prgm 1.py ======================
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\stant\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
enter the paragraph:NLP stands for Natural Language Processing. Computers process human language through NLP. NLP has great futu
e scope.
sentence tokenization:
['NLP stands for Natural Language Processing.', 'Computers process human language through NLP.', 'NLP has great future scope.']

word tokenization:
['NLP', 'stands', 'for', 'Natural', 'Language', 'Processing', '.', 'Computers', 'process', 'human', 'language', 'through', 'NLP',
'.', 'NLP', 'has', 'great', 'future', 'scope', '.']
```

Program No: 2

Aim

Build a simple unigram language model for a given corpus.

Program

```
from collections import Counter
corpus=input("enter the text from corpus:")
words=corpus.lower().split()
word_count=Counter(words)
total_words=len(words)
print("unigrams:")
for word in words:
    print(word)
```

Output

```
>>>
====================== RESTART: D:\MMM\NLP Lab\Prgm 2.py ======================
enter the text from corpus:Natural Language Processing is interesting and Natural Language Processing is useful
unigrams:
natural
language
processing
is
interesting
and
natural
language
processing
is
useful
```

Program No: 3

Aim

Implement a rule-based system to identify parts of a sentence.

Program

```
sentence=input("enter a sentence:")
words=sentence.lower().split()
print("\nparts of sentence:")
for word in words:
    if word in["a","an","the"]:
        print(word,"->Article")
    elif word.endswith("ing"):
        print(word,"->Verb")
    elif word.endswith("ly"):
        print(word,"->Adverb")
    elif word.endswith(("ous","ful","ive")):
        print(word,"->Adjective")
    else:
        print(word,"->Noun")
```

Output

```
>>>
======================== RESTART: D:\MMM\NLP Lab\Prgm 3.py ========================
enter a sentence:A powerful machine is processing data quickly.

parts of sentence:
a ->Article
powerful ->Adjective
machine ->Noun
is ->Noun
processing ->Verb
data ->Noun
quickly. ->Noun
```

Program No: 4

Aim

Perform basic semantic analysis using word similarity.

Program

```
import nltk
from nltk.corpus import wordnet
nltk.download('wordnet')
word1=input("enter first word:")
word2=input("enter second word:")
syn1=wordnet.synsets(word1)[0]
syn2=wordnet.synsets(word2)[0]
similarity=syn1.wup_similarity(syn2)
if similarity>=0.8:
    level="Highly Similar"
elif similarity>=0.5:
    level="Moderately Similar"
else:
    level="Less Similar"
print("word similarity:",similarity)
print("result:",level)
```

Output

```
>>>
======================== RESTART: D:\MMM\NLP Lab\prgm 4.py ========================
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\stant\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
enter first word:car
enter second word:vehicle
word similarity: 0.8
result: Highly Similar
```

Program No: 5

Aim

Demonstrate pragmatics by identifying speech acts (question, command, statement).

Program

```
sentence=input("enter a sentence:")
if sentence.endswith("?"):
    print("Speech Act:Question")
elif sentence.lower().startswith(("please","go","do","open")):
    print("Speech Act:Command")
else:
    print("Speech Act:Statement")
```

Output

```
>>>
========== RESTART: D:\MMM\NLP Lab\prgm 5.py ==========
enter a sentence:open door
Speech Act:Command
```

Program No: 6

Aim

Use Python libraries to analysis speech signals.

Program

```
import nltk
import librosa
audio,sr=librosa.load("sample.wav")
print("Sample Rate:",sr)
print("Audio Length:",len(audio))
```

Output

```
>>>
========== RESTART: D:\MMM\NLP Lab\prgm 6.py ==========
Sample Rate: 22050
Audio Length: 131047
```

Program No: 7

Aim

Convert text to phoneme representation using a phonetic dictionary.

Program

```
import nltk
from nltk.corpus import cmudict
nltk.download('cmudict')
dictionary=cmudict.dict()
word=input("enter word:").lower()
if word in dictionary:
    print("phonemes:",dictionary[word][0])
else:
    print("word not found")
```

Output

```
>>>
========================================= RESTART: D:\MMM\NLP LAB\prgm 7.py ========
[nltk_data] Downloading package cmudict to
[nltk_data]     C:\Users\stant\AppData\Roaming\nltk_data...
[nltk_data]   Package cmudict is already up-to-date!
enter word:Language
phonemes: ['L', 'AE1', 'NG', 'G', 'W', 'AH0', 'JH']
```

Program No: 8

Aim

Write a program to demonstrate ASCII vs Unicode encoding.

Program

```
text=input("enter the text:")
print("ASCII:")
for ch in text:
    print(ch,ord(ch))
print("\nUnicode:")
for ch in text:
    print(ch,ord(ch))
```

Output

```
>>>
========== RESTART: D:\MMM\NLP LAB\prgm 8.py ==========
enter the text:NLP
ASCII:
N 78
L 76
P 80

Unicode:
N 78
L 76
P 80
```

Program No: 9

Aim

Handle multilingual text input and normalization in Python.

Program

```
import unicodedata
text=input("Enter text:")
text=unicodedata.normalize('NFC',text)
text=text.lower()
text=text.strip()
print("Normalized Text:",text)
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 9.py ================
Enter text:Hello ന്ദ്മ്യാരാo
Normalized Text: hello ന്ദ്മ്യാരാo
```

Program No: 10

Aim

Implement text preprocessing: tokenization, stopword removal, stemming, lemmatization.

Program

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
text = input("Enter the text: ")
tokens = word_tokenize(text)
stop = stopwords.words('english')
filtered = []
for w in tokens:
    if w.lower() not in stop:
        filtered.append(w)
stemmer = PorterStemmer()
stemmed = []
for w in filtered:
    stemmed.append(stemmer.stem(w))
lemmatizer = WordNetLemmatizer()
lemmatized = []
```

```
for w in filtered:
    lemmatized.append(lemmatizer.lemmatize(w))
print("Tokens:", tokens)
print("Stopword Removed:", filtered)
print("Stemmed:", stemmed)
print("Lemmatized:", lemmatized)
```

Output

```
>>>
========== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/10.py ==========
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Enter the text: NLP stands for Natural Language Processing.
Tokens: ['NLP', 'stands', 'for', 'Natural', 'Language', 'Processing', '.']
Stopword Removed: ['NLP', 'stands', 'Natural', 'Language', 'Processing', '.']
Stemmed: ['nlp', 'stand', 'natur', 'languag', 'process', '.']
Lemmatized: ['NLP', 'stand', 'Natural', 'Language', 'Processing', '.']
```

Program No: 11

Aim

Load and explore a corpus using NLTK.

Program

```
import nltk
from nltk.corpus import gutenberg
nltk.download('gutenberg')
print("Available Corpus Files:")
print(gutenberg.fileids())
text = gutenberg.words('austen-emma.txt')
print("\nFirst 20 Words:")
print(text[:20])
print("\nTotal Words:")
print(len(text))
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 11.py ================
[nltk_data] Downloading package gutenberg to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\gutenberg.zip.
Available Corpus Files:
['austen-emma.txt', 'austen-persuasion.txt', 'austen-sense.txt', 'bible-kjv.txt', 'blake-poems.txt', 'bryan
t-stories.txt', 'burgess-busterbrown.txt', 'carroll-alice.txt', 'chesterton-ball.txt', 'chesterton-brown.tx
t', 'chesterton-thursday.txt', 'edgeworth-parents.txt', 'melville-moby_dick.txt', 'milton-paradise.txt', 's
hakespeare-caesar.txt', 'shakespeare-hamlet.txt', 'shakespeare-macbeth.txt', 'whitman-leaves.txt']

First 20 Words:
['[', 'Emma', 'by', 'Jane', 'Austen', '1816', ']', 'VOLUME', 'I', 'CHAPTER', 'I', 'Emma', 'Woodhouse', ',',
'handsome', ',', 'clever', ',', 'and', 'rich']

Total Words:
192427
```

Program No: 12

Aim

Access and query WordNet for synonyms, antonyms and hypernyms.

Program

```
import nltk
from nltk.corpus import wordnet
nltk.download('wordnet')
word = input("Enter a word: ").lower()
synsets = wordnet.synsets(word)
if not synsets:
    print("Word not found in WordNet")
else:
    synonyms = set()
    antonyms = set()
    hypernyms = set()
    for syn in synsets:
        for lemma in syn.lemmas():
            synonyms.add(lemma.name())
            for ant in lemma.antonyms():
                antonyms.add(ant.name())
        for h in syn.hypernyms():
            hypernyms.add(h.name().split('.')[0])
```

```
print("\nSynonyms:")
print(*list(synonyms)[:1], sep="\n")
print("\nAntonyms:")
if antonyms:
    print(*list(antonyms)[:1], sep="\n")
else:
    print("No antonyms found")
print("\nHypernyms:")
if hypernyms:
    print(*list(hypernyms)[:1], sep="\n")
else:
    print("No hypernyms found")
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 12.py ==================
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Enter a word: Good

Synonyms:
near

Antonyms:
badness

Hypernyms:
advantage
```

Program No: 13

Aim

Parse and extract information from an XML based linguistic resource.

Program

Data.xml

```
<corpus>
   <sentence id="1">
      <word>Natural</word>
      <word>Language</word>
      <word>Processing</word>
   </sentence>
   <sentence id="2">
      <word>XML</word>
      <word>parsing</word>
      <word>is</word>
      <word>easy</word>
   </sentence>
</corpus>
```

Program.py

```
import xml.etree.ElementTree as ET
tree = ET.parse("data.xml")
root = tree.getroot()
```

```
for sentence in root.findall("sentence"):
    sid = sentence.get("id")
    print("\nSentence ID:", sid)
    words = []
    for word in sentence.findall("word"):
        words.append(word.text)
    print("Words:", words)
    print("Sentence:", " ".join(words))
    words.append(word.text)
    print("Words:","".join(words))
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 13.py ================

Sentence ID: 1
Words: ['Natural', 'Language', 'Processing']
Sentence: Natural Language Processing
Words: NaturalLanguageProcessingProcessing

Sentence ID: 2
Words: ['XML', 'parsing', 'is', 'easy']
Sentence: XML parsing is easy
Words: XMLparsingiseasyeasy
```

Program No: 14

Aim

Write regx patterns to extract emails, dates, and URLs from text.

Program

```
import re

text = input("Enter text: ")

email_pattern = r'[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+'

emails = re.findall(email_pattern, text)

print("Emails Found:")

print(emails)

url_pattern = r'https?://[^\s]+'

urls = re.findall(url_pattern, text)

print("\nURLs Found:")

print(urls)

date_pattern = r'\d{2}[/\-]\d{2}[/\-]\d{4}|\d{4}-\d{2}-\d{2}'

dates = re.findall(date_pattern, text)

print("\nDates Found:")

print(dates)
```

Output

```
>>>
============================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 14.py ==============================
Enter text: Contact us at support@gmail.com or admin123@university.edu. Website: https://www.google.com and
http://nlp.example.org Important dates: 12/06/2026, 2026-06-13, 01-01-2025
Emails Found:
['support@gmail.com', 'admin123@university.edu.']

URLs Found:
['https://www.google.com', 'http://nlp.example.org']

Dates Found:
['12/06/2026', '2026-06-13', '01-01-2025']
```

Program No: 15

Aim

Implement a simple Finite State Automation (FSA) for word recognition.

Program

```
word=input("Enter a word:").lower()
state=0
for ch in word:
    if 'a'<=ch<='z':
        state=1
    else:
        state=-1
        break
if state==1:
    print("Accepted (Valid word)")
else:
    print("Rejected (Invalid word)")
```

Output

```
>>>
    ================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 15.py ========
    Enter a word:Hello
    Accepted (Valid word)
    Enter a word:hello123
    Rejected (Invalid word)
```

Program No: 16

Aim

Implement simple morphological analyser (prefix, suffix stripping).

Program

```
word = input("Enter a word: ").lower()
original = word
prefixes = [
    "un", "re", "in", "im", "il", "ir", "dis", "mis", "non",
    "pre", "post"
]
suffixes = [
    "ing", "ed", "er", "est", "ly", "ment", "ness", "ful", "less",
    "s", "es"
]
prefix_found = []
suffix_found = []
changed = True
while changed:
    changed = False
    for pre in prefixes:
        if word.startswith(pre) and len(word) > len(pre) + 2:
            prefix_found.append(pre)
```

```
            word = word[len(pre):]
            changed = True
            break
    changed = True
    while changed:
        changed = False
        for suf in suffixes:
            if word.endswith(suf) and len(word) > len(suf) + 2:
                suffix_found.append(suf)
                word = word[:-len(suf)]
                changed = True
                break
    print("\nOriginal Word:", original)
    print("Prefixes removed:", prefix_found if prefix_found else "None")
    print("Suffixes removed:", suffix_found if suffix_found else "None")
    print("Root Word:", word)
```

Output

```
>>>
    ================= RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 16.py =====
    Enter a word:reworking

    Original Word: reworking
    Prefixes removed: ['re']
    Suffixes removed: ['ing']
    Root Word: work
```

Program No: 17

Aim

Perform POS tagging using NLTK.

Program

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
text=input("Enter a Sentence:")
words=word_tokenize(text)
pos_tags=nltk.pos_tag(words)
print("\nPOS Tags:")
for word,tag in pos_tags:
    print(word,"->",tag)
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/Prgm 20.py ==========
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
Enter a Sentence:John is studying NLP.

POS Tags:
John -> NNP
is -> VBZ
studying -> VBG
NLP -> NNP
. -> .
```

Program No: 18

Aim

Implement a stochastic POS tagger using Hidden Markov Model (HMM).

Program

```
import nltk
from nltk.corpus import treebank
from nltk.tokenize import word_tokenize
from nltk.tag import DefaultTagger, UnigramTagger, BigramTagger
nltk.download('treebank')
nltk.download('punkt')
train_data = treebank.tagged_sents()
default_tagger = DefaultTagger('NN')
unigram = UnigramTagger(train_data, backoff=default_tagger)
bigram = BigramTagger(train_data, backoff=unigram)
sentence = input("Enter a sentence: ")
words = word_tokenize(sentence)
result = bigram.tag(words)
print("\nPOS Tagged Output:")
for word,tag in result:
    print(word,"->",tag)
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 21.py =======
[nltk_data] Downloading package treebank to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package treebank is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a sentence: NLP is very interesting

POS Tagged Output:
NLP -> NN
is -> VBZ
very -> RB
interesting -> JJ
```

Program No: 19

Aim

Demonstrate Transformation Based Learning (TBL) tagging.

Program

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')
text = input("Enter sentence: ")
words = word_tokenize(text)
tags = pos_tag(words)
transformed = []
for word, tag in tags:
    if word.lower().endswith("ing"):
        tag = "VBG"
    elif word.lower().endswith("ly"):
        tag = "RB"
    elif word[0].isupper():
        tag = "NNP"
    elif word.isdigit():
        tag = "CD"
    transformed.append((word, tag))
print("\nTagged Output:")
for word, tag in transformed:
    print(word, "->", tag)
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 22.py ========
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
Enter sentence: John plays football

Tagged Output:
John -> NNP
plays -> VBZ
football -> NN
```

Program No: 20

Aim

Handle unknown words in POS tagging.

Program

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
text = input("Enter sentence: ")
words = word_tokenize(text)
for word in words:
    tag = pos_tag([word])[0][1]
    if tag == 'NN':
        if word.endswith("ing"):
            tag = "VBG"
        elif word.endswith("ly"):
            tag = "RB"
        elif word[0].isupper():
            tag = "NNP"
        elif word.isdigit():
            tag = "CD"
    print(word, "->", tag)
```

Output

```
>>>
================= RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 23.py =========
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
Enter sentence: Riya works in Xyzon company
Riya -> NNP
works -> NNS
in -> IN
Xyzon -> NNP
company -> NN
```

Program No: 21

Aim

Define a Context Free Grammer (CFG) in Python and parse sentences.

Program

```
import nltk
from nltk import CFG
from nltk.parse import ChartParser
grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N | Det Adj N | 'I' | 'You'
VP -> V NP | V
Det -> 'a' | 'an' | 'the'
Adj -> 'big' | 'small' | 'red'
N -> 'cat' | 'dog' | 'apple' | 'boy' | 'girl'
V -> 'eat' | 'see' | 'like' | 'likes'
""")
parser = ChartParser(grammar)
sentence = input("Enter a sentence: ")
words = sentence.split()
words = sentence.split()
valid_words = ["I", "You","a", "an", "the","big", "small", "red","cat","dog", "apple", "boy", "girl","eat", "see", "like", "likes"]
if all(word in valid_words for word in words):
    trees = list(parser.parse(words))
    if trees:
        print("\nSentence follows the CFG!\n")
```

```
        for tree in trees:
            print(tree)
            tree.pretty_print()
    else:
        print("\nSentence does NOT follow the CFG.")
else:
    print("\nSentence contains words not in grammar.")
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 25.py ==============
Enter a sentence: the big dog likes a cat

Sentence follows the CFG!

(S
  (NP (Det the) (Adj big) (N dog))
  (VP (V likes) (NP (Det a) (N cat))))
          S
      ____|_________
     |             VP
     |          ____|___
     NP        |        NP
  ___|___      |     ___|___
Det Adj  N     V   Det      N
 |   |   |     |    |       |
the big dog likes  a      cat
```

Program No: 22

Aim

Use chart parsing to analyse ambiguous sentences.

Program

```
import nltk
from nltk import CFG
from nltk.parse import ChartParser
grammar = CFG.fromstring("""
S -> NP VP
NP -> Pronoun
NP -> Det N
NP -> Det N PP
VP -> V NP
VP -> VP PP
PP -> P NP
Pronoun -> 'I'
Det -> 'the'
N -> 'man' | 'telescope'
V -> 'saw'
P -> 'with'
""")
sentence = "I saw the man with the telescope"
tokens = sentence.split()
parser = ChartParser(grammar)
print("Possible Parse Trees:\n")
for tree in parser.parse(tokens):
```

```
print(tree)

print("\n")

tree.pretty_print()
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 27.py ======
Possible Parse Trees:

(S
  (NP (Pronoun I))
  (VP
    (VP (V saw) (NP (Det the) (N man)))
    (PP (P with) (NP (Det the) (N telescope)))))


          S
      ____|___________
     |               VP
     |        _______|_____
     |       VP             PP
     |    ___|___         ___|___
     |   NP      |       NP      |
     |   |       |    ___|___    |      ___|_____
Pronoun  V     Det   N      P   Det          N
     |   |      |    |      |    |           |
     I   saw    the  man    with the        telescope

(S
  (NP (Pronoun I))
  (VP
    (V saw)
    (NP (Det the) (N man) (PP (P with) (NP (Det the) (N telescope))))))


          S
      ____|_____
     |          VP
     |     _____|___
     |    |         NP
     |    |     ____|____
     |    |    |    |    PP
     |    |    |    |    |
     |    |    |    |    |         NP
     |    |    |    |    |      ___|_____
Pronoun   V   Det   N    P     Det        N
     |    |    |    |    |     |          |
     I    saw  the  man  with   the      telescope
```

Program No: 23

Aim

Load and analyze TreeBank using NLTK.

Program

```
import nltk
from nltk.corpus import treebank
nltk.download('treebank')
sentences = treebank.tagged_sents()
print("Total sentences:", len(sentences))
print("\nFirst sentence:")
print(sentences[0])
words = treebank.words()
print("\nTotal words:", len(words))
print("\nFirst 10 words:")
print(words[:10])
tags = [tag for (word, tag) in treebank.tagged_words()]
freq = nltk.FreqDist(tags)
print("\nMost common POS tags:")
print(freq.most_common(5))
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 29.py ================
[nltk_data] Downloading package treebank to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package treebank is already up-to-date!
Total sentences: 3914

First sentence:
[('Pierre', 'NNP'), ('Vinken', 'NNP'), (',', ','), ('61', 'CD'), ('years', 'NNS'), ('old', 'JJ'), (',', ','
), ('will', 'MD'), ('join', 'VB'), ('the', 'DT'), ('board', 'NN'), ('as', 'IN'), ('a', 'DT'), ('nonexecutiv
e', 'JJ'), ('director', 'NN'), ('Nov.', 'NNP'), ('29', 'CD'), ('.', '.')]

Total words: 100676

First 10 words:
['Pierre', 'Vinken', ',', '61', 'years', 'old', ',', 'will', 'join', 'the']

Most common POS tags:
[('NN', 13166), ('IN', 9857), ('NNP', 9410), ('DT', 8165), ('-NONE-', 6592)]
```

Program No: 24

Aim

Represent sentence meaning using logical forms.

Program

```
sentence = input("Enter a sentence: ")
words = sentence.lower().replace(".", "").split()
if len(words) < 2:
    print("Not enough words to form a logical representation")
else:
    subject = words[0]
    verb = words[1]
    if len(words) > 2:
        obj = "_".join(words[2:])
        logical_form = f"{verb}({subject}, {obj})"
    else:
        logical_form = f"{verb}({subject})"
    print("\nLogical Form:")
    print(logical_form)
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 32.py ==============
Enter a sentence: The boy eats an apple.

Logical Form:
boy(the, eats_an_apple)
```

Program No: 25

Aim

Build a simple Machine Translation System using sequence models.

Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
english =["i am student",
    "how are you",
    "i love machine learning",
    "what is your name",
    "good morning",
    "i am fine",
    "where are you going",
    "i need help",
    "thank you",
    "good night"]
malayalam =["ഞാൻ വിദ്യാർത്ഥി ആണ്",
    "സുഖമാണോ",
    "എനിക്ക് മെഷീൻ ലേണിംഗ് ഇഷ്ടമാണ്",
    "നിങ്ങളുടെ പേര് എന്താണ്",
    "സുപ്രഭാതം",
    "എനിക്ക് സുഖമാണ് ",
    "നിങ്ങൾ എവിടേക്ക് പോകുന്നു",
    "എനിക്ക് സഹായം വേണം",
```

```
    "നന്ദി",
    "ശുഭ രാത്രി"]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(english)
def translate(sentence):
    sentence_vec = vectorizer.transform([sentence])
    similarity = cosine_similarity(sentence_vec, X)
    index = similarity.argmax()
    score = similarity[0][index]
    if score < 0.2:
        return "Translation not found in dataset"
    return malayalam[index]
while True:
    text = input("\nEnter English sentence (or 'exit'): ").lower()
    if text == "exit":
        break
    print("Malayalam Translation:", translate(text))
```

Output

```
>>>
================== RESTART: D:/MMM/SAC/S5 BCA NLP/Lab/prgm 39.py ======

Enter English sentence (or 'exit'): Good Morning
Malayalam Translation: സുപ്രഭാതം

Enter English sentence (or 'exit'): exit
```